
Generating Effective Alpha Feature Collections via Deep Reinforcement Learning

Yuteng Zhuang
Stanford University
yutengz2@stanford.edu

Shuchen(James) Liu
Stanford University
shuchen@stanford.edu

1 Introduction

In quantitative finance, the ability to extract meaningful and predictive signals from raw market data is critical for building profitable trading strategies. These signals, known as alpha factors, help identify market trends, forecast returns, and inform portfolio management decisions. Alpha factors are particularly valuable for their potential to deliver excess returns, making their discovery a key focus for researchers and practitioners alike. Alpha factors can broadly be classified into two categories: deep learning-based models and formulaic representations. While deep models excel in capturing complex relationships within data, their black-box nature limits interpretability. This lack of transparency poses significant challenges in financial applications, where practitioners must understand and manage risks. As a result, formulaic alpha factors, which express financial principles in concise mathematical forms, are favored in practice because of their interpretability and ease of analysis.

Despite their advantages, discovering high-quality formulaic alpha factors remains a challenging task. Traditional methods such as genetic programming is computationally expensive and inefficient in large search spaces. Recent advances have introduced reinforcement learning (RL) approaches to address these challenges and have shown promise in exploring large search spaces.

This project proposes a framework to mine robust formulaic alpha factors by combining deep reinforcement learning with tree-based predictive models. The framework comprises two modules: a Generator and a Predictor. The Generator, powered by Proximal Policy Optimization (PPO), generates a pool of formulaic alpha factors in reverse Polish notation (RPN). The Predictor uses LightGBM to evaluate and fit these factors, providing feedback to the Generator through Information Ratio (IR)-based reward signals. This iterative process enables the system to refine alpha factor sets that maximize predictive synergy while preserving interpretability. The input to our framework consists of six core financial features derived from historical stock data: open, close, high, low, volume, and vwap (volume-weighted average price). The output of the model consists of a pool of alpha factors that maximize predictive performance and robustness. Additionally, the model produces a trained LightGBM model that has been fitted using these factors.

2 Related work

Previous research by Yihuang Guo and Rui Zhao [1] demonstrated the strong predictive capability of deep learning-based approaches in alpha factor mining. However, these methods suffer from a lack of interpretability, which limits their applicability in risk-sensitive financial markets due to their black-box nature. In contrast, formulaic alphas—historically crafted by human experts—encode clear economic principles and offer much-needed transparency. For instance, Kakushadze introduced the "101 Formulaic Alphas" [2] in 2016 and demonstrated their effectiveness in the U.S. market. However, manually designing formulaic alphas is a time-intensive process that depends heavily on expert knowledge. To overcome these challenges, researchers have developed algorithms to automate the discovery of formulaic alpha factors. These methods include tree-based models as well as heuristic techniques such as Genetic Programming [3-4]. More recently, a deep reinforcement learning-based

framework [5] for alpha factor mining was introduced, demonstrating superior performance compared to both tree-based methods and Genetic Programming, while retaining good interpretability. Building on this foundation, this project proposes an enhanced framework that combines deep reinforcement learning with tree-based predictive models. This integrated approach aims to further improve the quality and synergy of generated formulaic alphas while addressing existing challenges such as overfitting and noise in the training process.

3 Dataset and Features

This project uses stock data from the CSI 100, CSI 300, and CSI 500 indices, which are well-known benchmarks for the Chinese stock market. These indices were chosen to align with datasets used in existing research, ensuring that the proposed framework’s performance can be fairly compared against prior methods on the same asset types and time periods. Demonstrating strong results on these datasets establishes the framework’s effectiveness, after which we plan to extend the evaluation to other markets, such as U.S. stock data, to assess its ability to generalize across different financial environments. The CSI indices represent a broad spectrum of the Chinese stock market. The CSI 100 tracks the largest 100 blue-chip companies, the CSI 300 covers the top 300 large- and mid-cap firms, and the CSI 500 represents the next 500 smaller, growth-oriented companies. Together, they reflect the performance of China’s stock market across different company sizes. The dataset spans daily stock-level data over multiple years, with raw features that include open, close, high, low, volume, and VWAP (volume-weighted average price).

The dataset spans daily stock-level data from 1990 to 2024, covering a total of 5,769 stocks with raw features including *open*, *close*, *high*, *low*, *volume*, and *VWAP* (volume-weighted average price). In this milestone, the dataset is split by date into a training set from 2016-01-01 to 2022-05-31, a validation set from 2022-06-01 to 2023-05-31, and a test set from 2023-06-01 to 2024-05-31. The input tensor has dimensions:

$$\mathbf{X} \in \mathbb{R}^{d \times f \times s},$$

where d is the number of trading days within this period, f is the number of raw features (open, close, high, low, volume, VWAP), and s is the number of stocks in the CSI 500 index. Each dataset contains $s \cdot d$ examples. Specifically, the training set contains $1687 \cdot 500 = 843,500$ examples, the validation set contains $252 \cdot 500 = 126,000$ examples, and the test set contains $254 \cdot 500 = 127,000$ examples.

4 Methods

Proximal Policy Optimization (PPO) is a reinforcement learning algorithm designed to balance exploration and exploitation while ensuring stable training. PPO works by optimizing a clipped surrogate objective function, which constrains the updates to the policy within a trust region. The algorithm uses a ratio $r_t(\theta)$ that compares the probability of taking an action under the updated policy $\pi_\theta(a_t | s_t)$ to the probability under the old policy $\pi_{\text{old}}(a_t | s_t)$. This ratio is expressed as:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

To stabilize training, PPO maximizes the minimum of the unclipped objective and a clipped version of it. The clipped objective ensures that $r_t(\theta)$ stays within a safe range, typically $[1 - \epsilon, 1 + \epsilon]$, where ϵ is a small constant (e.g., 0.2). The surrogate objective $L_{\text{CLIP}}(\theta)$ is defined as:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)],$$

where A_t is the advantage function, representing how much better an action is compared to the policy’s baseline. By clipping the ratio, PPO discourages large updates that could destabilize the learning process.

This project uses a Maskable PPO, which extends the standard PPO framework by integrating an "invalid action masking" mechanism. This ensures that only valid actions are considered at each decision step. In each state, a mask is applied to filter out actions that would violate predefined rules, effectively narrowing the policy’s focus to feasible choices. By selectively sampling from the valid action space, maskable PPO accelerates learning, reduces exploration inefficiencies, and

ensures adherence to constraints, making it especially effective in tasks with complex and rule-bound action spaces. This property is particularly well-suited for the formulaic alpha generation task in this project. The use of the clipped surrogate objective function ensures that no excessively large rewards or penalties are applied during training, which could otherwise lead to overfitting or failure of the network parameters to converge. This approach is particularly valuable for navigating stock market datasets, which are inherently noisy and prone to volatility.

A) Generator

Transformer is used as a feature extractor, leveraging its attention mechanism to capture long-range dependencies and complex patterns in the input data. The Transformer processes raw input features—in our case, they are open, close, high, low, volume, and vwap—to generate rich feature representations. Originally designed for machine translation tasks, the Transformer architecture consists of an encoder and a decoder, where the encoder processes sentences in the source language, and the decoder generates sentences in the target language. However, since this task does not involve sequence-to-sequence mapping, we only need to encode observation sequences for feature extraction, and therefore, the decoder component is not employed.

Building on the features extracted by the Transformer, both the value network and policy network share a common feature extractor that transforms the encoded token sequences into dense vector representations. These feature vectors are then fed into an MLP extractor. The state s_t at time step t represents the sequence of tokens generated thus far. Each state begins with a special start token (*BEG*) and is extended sequentially with operator, feature, or constant tokens. To ensure interpretability, the maximum length of any sequence is capped at 20 tokens. This design prioritizes human-readable alpha formulas while maintaining sufficient flexibility to capture meaningful patterns in stock data.

At each step, the action a_t selects a token to append to the current state. The set of valid actions is dynamically determined to ensure that the resulting sequence remains a valid RPN expression. For instance, unary operators (e.g., *log*) can only follow sequences with sufficient operands, while binary operators (e.g., ***) require at least two preceding operands. Invalid actions are masked, restricting the agent to a constrained action space.

The transition from state s_t to s_{t+1} is deterministic. The next state is constructed by appending the selected action a_t to the current sequence. Once a valid formula is completed (indicated by a special end token, *SEP*), the episode terminates. The generator parses the final state into a formulaic alpha expression, which is subsequently evaluated for its predictive performance in the Predictor module.

B) Predictor

The objective of the Predictor module is to predict stock returns using the alpha factor pool generated by the Generator, evaluate its performance by calculating its correlation with the actual stock returns, and provide a reward signal back to the Generator to update the PPO policy.

To evaluate the effectiveness of a formulaic alpha factor, the Information Coefficient (IC) is a widely used metric. The IC is defined as the Pearson correlation coefficient between the ground-truth asset price y_t and the factor value z_t . This metric quantifies the linear relationship between the predicted and actual values, providing an indicator of the factor’s predictive power. Mathematically, it is expressed as:

$$IC(z_t, y_t) = \frac{\text{Cov}(z_t, y_t)}{\sigma_{z_t} \sigma_{y_t}},$$

where $\text{Cov}(u, v)$ represents the covariance between two vectors u and v , and σ denotes the standard deviation. For this project, Information Ratio (IR) is used as the Reward Signal to train the RL policy. IR measures the return earned in excess of a benchmark (often referred to as active return) relative to the risk taken. Mathematically, it is expressed as:

$$IR = \frac{\mathbb{E}_t [IC(z_t, y_t)]}{\sqrt{\text{Var}(IC(z_t, y_t))}},$$

where:

- $\mathbb{E}_t [IC(z_t, y_t)]$ is the expected value of the Information Coefficient (IC).
- $\text{Var}(IC(z_t, y_t))$ is the variance of the Information Coefficient.

- z_t represents the alpha signal at time t .
- y_t represents the target variable at time t .

Using IR as the reward signal inherently balances the trade-off between return and risk. By optimizing IR, the reinforcement learning agent is incentivized to generate alpha factors that enhance predictive accuracy (returns) while minimizing performance variability (risk). This approach ensures that the alpha generator’s output aligns closely with the real-world financial metrics that practitioners prioritize.

Given a stock market dataset $\mathbf{X} \in \mathbb{R}^{n \times m \times \tau}$, representing n stocks with m raw features over τ trading days, the goal is to identify a set of k alpha factors $\mathbf{F} = \{f_1, f_2, \dots, f_k\}$. Each alpha factor f maps $\mathbf{X}$ to $\mathbf{z} \in \mathbb{R}^n$, which is used as input to the LightGBM model.

The optimization objective is:

$$\max_{\mathbf{F}} \sigma_y (c^* (\mathbf{X}; \mathbf{F})),$$

where:

- $c^* (\mathbf{X}; \mathbf{F})$ represents the LightGBM model trained on $\mathbf{F}$.
- σ_y is the function of Information Ratio (IR).

5 Experiments/Results/Discussion

Since good formulaic alpha factors of longer expressions are rarer than the shorter ones, the discount factor ($\gamma = 1$) ensures that long-term rewards are fully considered, encouraging the exploration of more complex and impactful alpha factors without prematurely penalizing longer expressions. The GAE ($\lambda = 0.95$) value balances the bias-variance tradeoff in advantage estimation, enabling stable policy updates while maintaining efficient learning. The clipping range (`clip_range = 0.2`) restricts the deviation between old and new policies, crucial for stabilizing updates in noisy financial environments. To promote exploratory behavior, the entropy coefficient (`ent_coef = 0.1`) introduces randomness in action selection, preventing the model from converging prematurely to suboptimal strategies. The batch size (256) and number of training steps per batch (`n_steps = 2048`) were chosen to ensure that enough data is processed per iteration for stable updates while fitting within memory constraints. Detailed hyper-parameter settings are provided in Appendix.

The model was trained on datasets corresponding to CSI100, CSI300, and CSI500 indices for the same time period. Since limited number of stocks are included in CSI100, the model exhibited pronounced overfitting when trained on this dataset. In contrast, training on CSI500 demonstrated better convergence, likely owing to the larger and more diverse dataset provided by this index. Considering computational resource constraints, subsequent analyses and experiments focus on the CSI300 and CSI500 indices to explore performance across a broader range of market conditions.

Table 1: Performance of Mined Factors on CSI300 and CSI500

Method	CSI300		CSI500	
	IC	RankIC	IC	RankIC
GP	0.0355	0.0573	0.0525	0.0565
Baseline PPO	-0.0038	0.0103	0.0439	0.0416
Modified Maskable PPO	-0.0010	-0.0129	0.0792	0.0588

The experimental results in Table 1 reveal differences in algorithm performance across the CSI300 and CSI500 datasets. Genetic Programming (GP) performed well on both CSI300 and CSI500, particularly on CSI500 (IC = 0.0525, RankIC = 0.0565). GP is effective at exploring large, non-linear search spaces, making it well-suited for discovering complex and meaningful alpha factors. However, the computational cost and inefficiency of maintaining a large population can hinder its scalability when handling multiple alpha factors simultaneously.

The proposed PPO framework achieved the highest performance on CSI500 by incorporating masking mechanisms, refining the reward function, and leveraging decision trees for factor combination within the framework. Gradient Boosting Decision Tree like LightGBM are powerful tools for capturing non-linear relationships, but they are very prone to overfitting. In detail, the PPO exploration process can generate noise factors (expressions with little true predictive value) in the early stages of iterations. Due to the high capacity of tree models, these noise factors can still achieve artificially high IR rewards, misleading the PPO framework into learning suboptimal policies. To solve this overfitting problem, I attempted to enhance the framework by introducing reward regularization to smooth IR signals, penalize unstable factors, and discard spurious factors during training. However, these techniques yielded limited improvements. Among the various approaches I tried, the most effective was to first pretrain a proxy network and an alpha pool using a linear combination model, and subsequently fine-tune the model. This two-step process proved to be more effective in improving performance compared to direct training.

It is important to note that a high IC value does not necessarily translate to high real-world trading performance. To address this, backtestings using the Qlib platform by Microsoft Research [9] were performed to simulate the trading signals generated by the model on historical data.

Table 2: Backtesting Results for Baseline PPO and Modified PPO on CSI500

Metric	Baseline PPO	Modified PPO
Sharpe Ratio	0.1746	0.5128
Annual Return	0.0300	0.1373
Maximum Drawdown	-0.1229	-0.1960
Annual Excess Return	0.1391	0.2463
Excess Maximum Drawdown	-0.1190	-0.1495

Table 2 shows the backtest results for baseline PPO and modified PPO in the CSI500 data set from 2023-06-01 to 2024-05-31 (test set). The detailed definitions and equations for these performance metrics are introduced in the Appendix for reference. In addition, the full benchmarks are provided in the Appendix. From the results, the proposed framework demonstrates strong performance on larger datasets like CSI500, but it struggles to produce effective factors on smaller datasets such as CSI100. This disparity can be attributed to mainly two reasons.

First, the CSI100 dataset contains a limited number of stocks, which reduces the diversity of the data. This lack of diversity restricts the model’s ability to generalize and identify robust, meaningful patterns. As a result, the factors mined from this smaller dataset are often less representative of broader market trends and are more likely to overfit to the specific characteristics of the included stocks. Second, smaller datasets like CSI100 inherently increase the risk of overfitting. With fewer data points available, the model may focus on capturing noise or idiosyncratic behaviors of individual stocks rather than extracting generalizable factors that could perform well across different market conditions.

6 Discussion

6.1 The other RL method

AlphaGen uses Proximal Policy Optimization (PPO) to update policy-value-entropy network. Policy head is used to predict next action; value head is used to evaluate the value of that action; and entropy loss is used to encourage exploration.

From the above Experiments, we observe that comparing to the total loss, the value loss fluctuate a lot [24]. Zhao et al. argue that during the scenario of Alpha mining, there is no non-terminal reward. Rewards are only given when the factor has established. This makes we do not have a lot of training data for value network comparing to policy network. Therefore, the value network is much harder to train and converge.

As the result, Zhao et al. proposed a new algorithm based on AlphaGen that use REINFORCE and baseline instead of PPO to train the model [24]. They also mathematically proved the advantage of this algorithm by following propositions:

- **Proposition 1:** The proposed gradient estimator with baseline is an unbiased estimator:

$$\mathbb{E}[\tilde{g}(\theta)] = \nabla_{\theta} \mathbb{E}_{a_{1:T} \sim \pi_{\theta}} [r(a_{1:T})].$$

- **Proposition 2:** With the policy distribution $p_{\theta}(a|a_{1:t-1})$ the variance of the gradient estimator is bounded by:

$$\text{Var}[\tilde{g}(\theta)] \leq \frac{8 \cdot r_{\max}^2 \cdot T^2}{N},$$

where $r_{\max}$ is the maximum reward, T is time length, and N is the number of samples.

- **Proposition 3:** Based on the above propositions, it is enough to show that this model has Lower Variance than traditional REINFORCE algorithm.

6.2 Interpretable factors generated by AlphaGen

We can have some insights from top 10 most frequent factors.

- Momentum factors are favored by AlphaGen. We can find many factors follow the structure of Rolling Sum or Rolling mean. This is a way to characterize the trends of fluctuation in prices.
- Volume is a important feature that describes these factors. Also, in the training step of the model, when we sample from the policy network, it is very often to generate *Volume* as factor. This shows the strong indicative nature of volume.

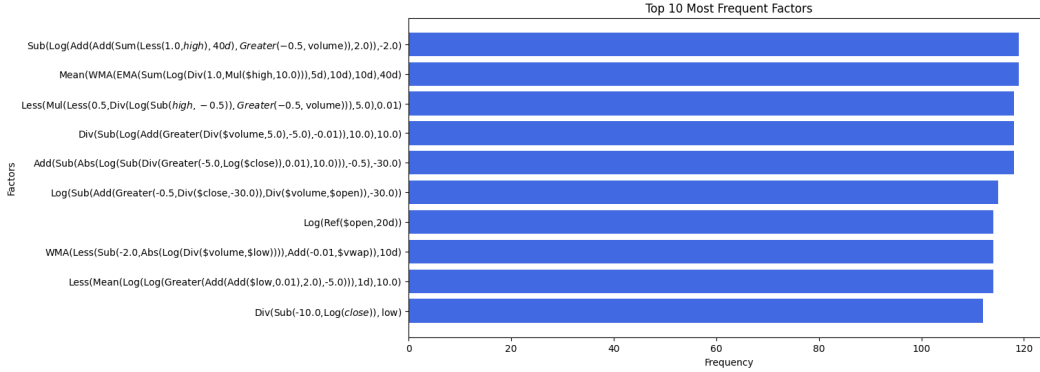


Figure 1: Top 10 Most Frequent Factors

6.3 Temporal Effectiveness of Factors

Table 3: Comparison of Factor Performance Over Time

Metric	2022-09-01 to 2022-12-30	2022-09-01 to 2023-12-30
Sharpe Ratio	1.1993	-1.8287
Annual Return	0.2717	-0.3409
Max Drawdown	-0.1229	-0.6478
Information Ratio	2.2496	-1.1950
Annual Excess Return	0.4074	0.2243
Excess Max Drawdown	-0.0565	-0.6288

In the process of backtesting, we also found that the temporal effectiveness of factors generated by Alphagen is very important. Most of the factors generated by AlphaGen are short-term factors and perform poorly in long run. From the above table, left column represents the short-term performance of the model; and the right column represents the long-term performance of the same model.

7 Conclusion/Future Work

This project explored mining formulaic alpha factors using a framework that combines Proximal Policy Optimization (PPO) with LightGBM. The proposed framework performed well on large datasets such as CSI500 and achieved strong results in backtesting on out-of-sample data. By using Information Ratio (IR) as the reward signal, the framework optimizes for long-term stability and risk-adjusted performance. In addition, the use of decision trees like LightGBM for modeling non-linear relationships is logical, as tree-based methods excel at capturing complex patterns. However, careful management is necessary to mitigate the risk of overfitting. For future work, introducing Monte Carlo to mock additional noise to the training data for data augmentation could be useful to improve the reliability and generalizability of the proposed framework.

8 Appendix

Evaluation Metrics

The **Sharpe Ratio** measures the risk-adjusted return of a portfolio by comparing its excess return (return above the risk-free rate) to its volatility. It is defined as:

$$\text{Sharpe Ratio} = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

where R_p is the portfolio return, R_f is the risk-free rate, and σ_p is the standard deviation of the portfolio returns. A higher Sharpe Ratio indicates better risk-adjusted performance.

The **Annual Return** represents the compounded growth rate of the portfolio over a year, indicating the strategy’s profitability. It is calculated as:

$$\text{Annual Return} = \left((1 + R_{\text{total}})^{\frac{1}{T}} \right) - 1$$

where R_{total} is the cumulative return and T is the number of years. This metric highlights the portfolio’s overall growth over time.

The **Maximum Drawdown (MDD)** measures the largest peak-to-trough decline in the portfolio value during a specified period. It is defined as:

$$\text{MDD} = \min \left(\frac{P_t - P_{\text{peak}}}{P_{\text{peak}}} \right)$$

where P_t is the portfolio value at time t and P_{peak} is the highest portfolio value up to time t . This metric is critical for assessing the portfolio’s risk and resilience during downturns.

The **Annual Excess Return** calculates the return of the portfolio over the benchmark on an annualized basis. It is given by:

$$\text{Annual Excess Return} = \text{Annual Return} - \text{Benchmark Return}$$

This metric shows how much the strategy outperforms the benchmark, reflecting its relative profitability.

The **Excess Maximum Drawdown** compares the drawdown of the portfolio to that of the benchmark. It is defined as:

$$\text{Excess MDD} = \text{MDD}_{\text{Portfolio}} - \text{MDD}_{\text{Benchmark}}$$

This metric indicates the additional risk borne by the portfolio relative to the benchmark during the worst drawdown.

Table 4: Hyper-parameter Settings

Parameter	Value	
pool_capacity	20	Limits the number of stored alpha factors for computational efficiency.
steps	50000	Ensures adequate training iterations for stable convergence without overtraining.
gamma	1	No discounting of future rewards to encourage exploration of long-term effective factors.
gae_lambda	0.95	Balances bias and variance for stable policy updates. (Default)
clip_range	0.2	Stabilizes updates by preventing large deviations between new and old policies. (Default)
ent_coef	0.15	Encourages exploration of less obvious alpha factors through entropy regularization.
n_steps	2048	Balances computational efficiency and learning stability per training batch. (Default)
batch_size	256	Mini-batch size for updates. Selected to balance learning stability and computational cost.
learning_rate	0.001	Low enough to ensure stable updates but high enough to converge within 50,000 steps.
n_epochs	10	Adequate number of epochs per batch for effective learning.
n_encoder_layers	6	Defines the depth of the Transformer encoder for capturing complex relationships.
d_model	128	Sets the width of each layer in the Transformer, balancing complexity and efficiency.
n_head	4	Allows capturing multi-head attention for diverse patterns in the data.
d_ffn	2048	Sets the hidden layer dimension to extract rich features.
dropout	0.1	Introduces regularization to reduce overfitting.

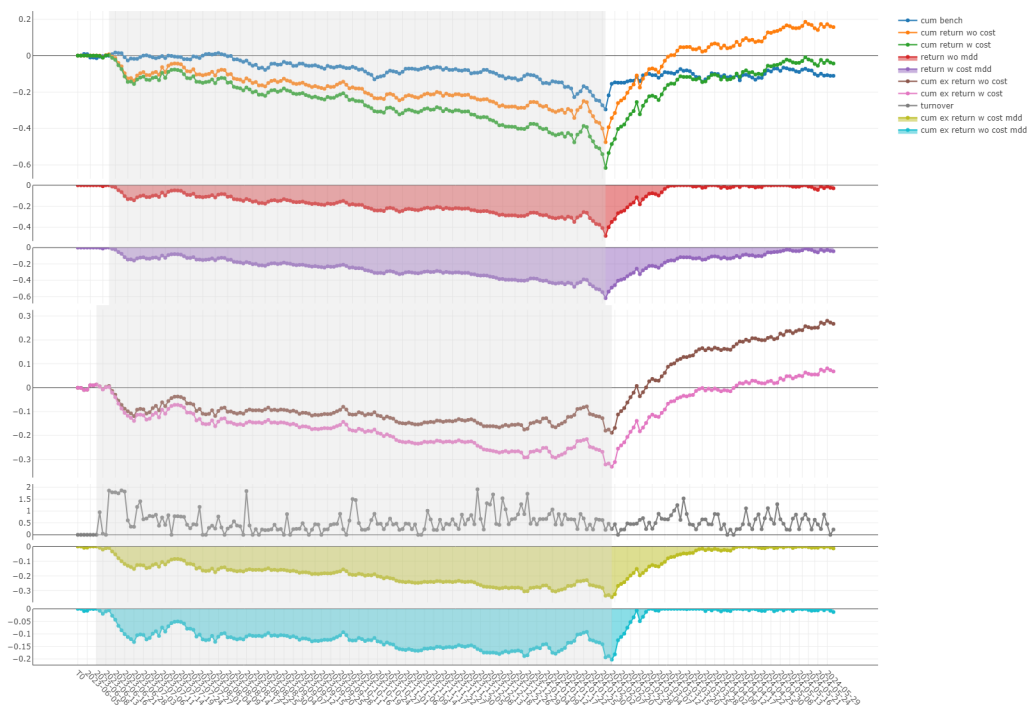


Figure 2: CSI500 Benchmark - Baseline PPO Framework

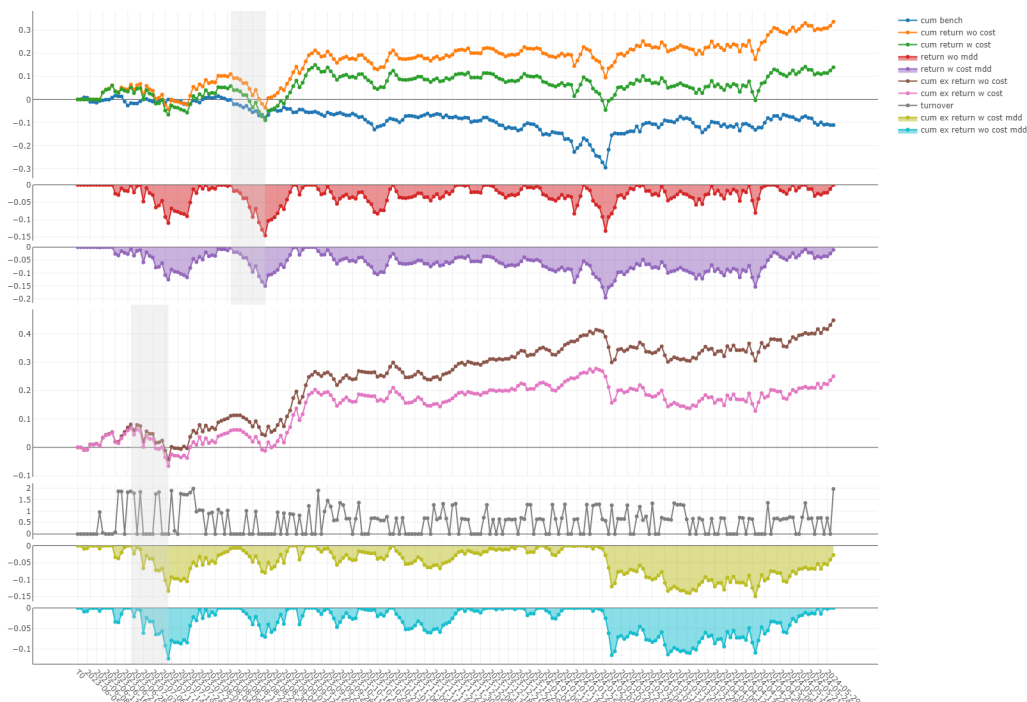


Figure 3: CSI500 Benchmark - Proposed PPO Framework

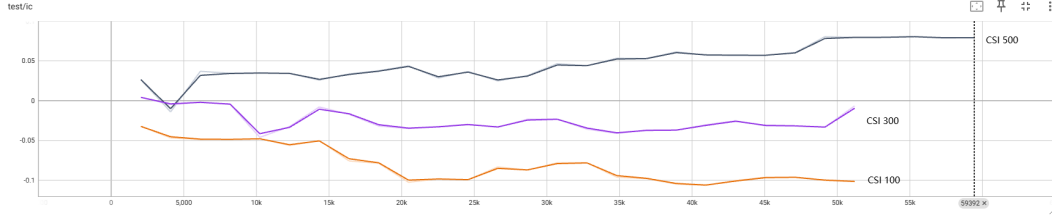


Figure 4: Test- IC

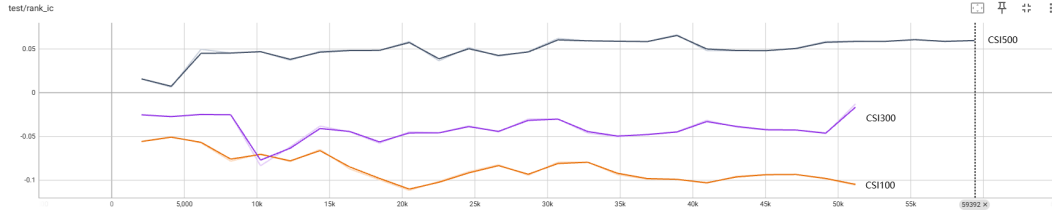


Figure 5: Test- Rank IC

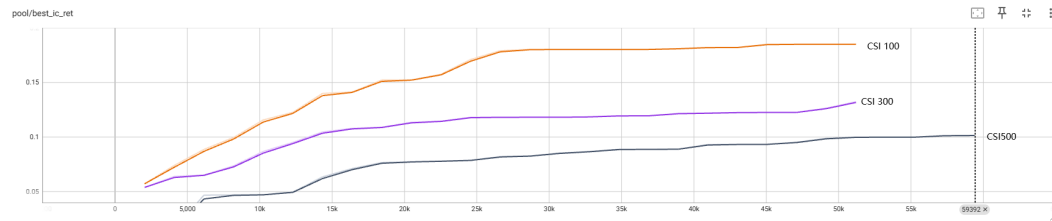


Figure 6: Validation- Best IC

References

- [1] Guo Y, Zhao R, Gou C. Mining Alpha Factors in Financial Markets using Multi-Frequency Timing Residual Networks[C]//2023 9th International Conference on Computer and Communications (ICCC). IEEE, 2023: 2508-2513.
- [2] Z. Kakushadze, "101 formulaic alphas," Wilmott, vol. 2016, no. 84, pp. 72–81, 2016.
- [3] Cui C, Wang W, Zhang M, et al. Alphaevolve: A learning framework to discover novel alphas in quantitative investment[C]//Proceedings of the 2021 International conference on management of data. 2021: 2208-2216.
- [4] Xiaoming Lin, Ye Chen, Ziyu Li, and Kang He. 2019. Revisiting Stock Alpha Mining Based On Genetic Algorithm. Technical Report. Huatai Securities Research Center.
- [5] Yu S, Xue H, Ao X, et al. Generating synergistic formulaic alpha collections via reinforcement learning[C]//Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2023: 5476-5486.
- [6] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.
- [7] Ke G, Meng Q, Finley T, et al. Lightgbm: A highly efficient gradient boosting decision tree[J]. Advances in neural information processing systems, 2017, 30.
- [8] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine learning in Python[J]. the Journal of machine Learning research, 2011, 12: 2825-2830.

- [9] Yang X, Liu W, Zhou D, et al. Qlib: An ai-oriented quantitative investment platform[J]. arXiv preprint arXiv:2009.11189, 2020.
- [10] Cui C, Wang W, Zhang M, et al. Alphaevolve: A learning framework to discover novel alphas in quantitative investment[C]//Proceedings of the 2021 International conference on management of data. 2021: 2208-2216.
- [11] Paszke A, Gross S, Massa F, et al. Pytorch: An imperative style, high-performance deep learning library[J]. Advances in neural information processing systems, 2019, 32.
- [12] Vaswani A. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017.
- [13] Zhao J, Zhang C, Qin M, et al. QuantFactor REINFORCE: Mining Steady Formulaic Alpha Factors with Variance-bounded REINFORCE[J]. arXiv preprint arXiv:2409.05144, 2024.
- [14] Z. Zhang, S. Zohren and S. Roberts, "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books," in IEEE Transactions on Signal Processing, vol. 67, no. 11, pp. 3001-3012, 1 June1, 2019, doi: 10.1109/TSP.2019.2907260.
- [15] Olorunnimbe, K., Viktor, H. Deep learning in the stock market—a systematic survey of practice, backtesting, and applications. *Artif Intell Rev* 56, 2057–2109 (2023). <https://doi.org/10.1007/s10462-022-10226-0>
- [16] I. Tulchinsky. 2015. Finding Alphas: A Quantitative Approach to Building Trading Strategies. 1–253 pages.
- [17] Zhicheng Wang, Biwei Huang, Shikui Tu, Kun Zhang, and Lei Xu. 2021. DeepTrader: A Deep Reinforcement Learning Approach for Risk-Return Balanced Portfolio Management with Market Conditions Embedding. In Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021. AAAI Press, 643–650.
- [18] Liheng Zhang, Charu C. Aggarwal, and Guo-Jun Qi. 2017. Stock Price Prediction via Discovering Multi-Frequency Trading Patterns. In Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017. ACM, 2141–2149.
- [19] Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. 2020. AutoAlpha: an Efficient Hierarchical Evolutionary Algorithm for Mining Alpha Factors in Quantitative Investment. arXiv preprint arXiv:2002.08245 (2020).
- [20] Shi H, Song W, Zhang X, et al. AlphaForge: A Framework to Mine and Dynamically Combine Formulaic Alpha Factors[J]. arXiv preprint arXiv:2406.18394, 2024.
- [21] M. . -J. Willis, H. G. Hiden, P. Marenbach, B. McKay and G. A. Montague, "Genetic programming: an introduction and survey of applications," Second International Conference On Genetic Algorithms In Engineering Systems: Innovations And Applications, Glasgow, UK, 1997, pp. 314-319, doi: 10.1049/cp:19971199.
- [22] S. Luke, "Two fast tree-creation algorithms for genetic programming," in IEEE Transactions on Evolutionary Computation, vol. 4, no. 3, pp. 274-283, Sept. 2000, doi: 10.1109/4235.873237. keywords: Genetic programming;Genetic mutations;Algorithm design and analysis;Functional programming;Optimization methods;Computer science;Size control;Tree data structures;Time factors,
- [23] Chen T, Guestrin C. Xgboost: A scalable tree boosting system[C]//Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining. 2016: 785-794.
- [24] Zhao J, Zhang C, Qin M, et al. QuantFactor REINFORCE: Mining Steady Formulaic Alpha Factors with Variance-bounded REINFORCE[J]. arXiv preprint arXiv:2409.05144, 2024.
- [25] Bahdanau D, Brakel P, Xu K, et al. An actor-critic algorithm for sequence prediction[J]. arXiv preprint arXiv:1607.07086, 2016.
- [26] Grondman I, Busoniu L, Lopes G A D, et al. A survey of actor-critic reinforcement learning: Standard and natural policy gradients[J]. IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews), 2012, 42(6): 1291-1307.